

TP N°1_Part1

Prise en main de l'environnement
Arduino et des entrées/sorties
numériques

Durée max conseillée : 3h



Partie lecture durée conseillée : 15 min

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant sera capable de :

- Installer et configurer l'IDE Arduino 2.x.
- Identifier les principaux éléments d'une carte Arduino Uno.
- Comprendre le rôle du microcontrôleur ATmega328P.
- Réaliser le câblage d'une maquette Grove.
- Compiler et téléverser un programme.
- Utiliser les fonctions fondamentales de l'API Arduino.
- Lire et commander des entrées/sorties numériques.
- Utiliser le Moniteur Série pour le débogage.
- Structurer un programme autour des fonctions `void setup(){}et void loop(){}et`

Prérequis

À l'issue d'enseignements précédents, l'étudiant est capable de :

- Informatique
 - Utiliser un ordinateur sous Windows 11.
 - Installer et utiliser un logiciel.
 - Gérer une arborescence de fichiers.
 - Manipuler des fichiers source et des bibliothèques.
- Algorithmique
 - Concevoir un algorithme simple.
 - Utiliser les structures conditionnelles (if, switch).
 - Utiliser les boucles (for, while, do...while).
 - Décomposer un problème en fonctions.
- Langage C/C++
 - Déclarer des variables et constantes.
 - Utiliser les principaux types de données.
 - Écrire et appeler des fonctions.
 - Manipuler des tableaux.
 - Comprendre les notions de portée des variables.
 - Utiliser les pointeurs (notions de base).
 - Compiler et corriger les erreurs de compilation.
- Programmation orientée objet
 - Comprendre les notions de classe et objet.
 - Utiliser un constructeur.
 - Déclarer des attributs et des méthodes.
 - Instancier un objet.

• Électronique

- Identifier les grandeurs électriques fondamentales (tension, courant).
- Lire un schéma électronique simple.
- Réaliser un câblage élémentaire.
- Identifier les principaux composants électroniques.

1.1 Introduction.

Avant de développer des applications embarquées complexes ou des objets connectés, il est indispensable de maîtriser les mécanismes fondamentaux permettant à un microcontrôleur d'interagir avec son environnement. Cette interaction repose sur les **entrées/sorties numériques (GPIO)**, les **entrées analogiques (CAN)** et les **sorties à modulation de largeur d'impulsion (PWM)**.

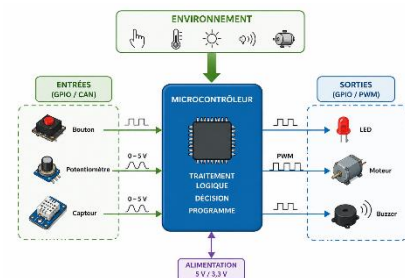
Au cours de ce TP, vous découvrirez comment acquérir des informations provenant de capteurs (boutons, potentiomètre), commander des actionneurs (LED, relais, moteurs etc) et échanger des informations avec l'utilisateur. Ces notions constituent la base de tout système embarqué moderne.

Les compétences acquises seront réinvesties tout au long de la formation pour développer des applications plus évoluées intégrant des capteurs, des afficheurs, des interfaces homme-machine et, dans la seconde partie du module, des objets connectés basés sur un ESP32.

1.1- Exemples d'applications

Les techniques étudiées dans ce TP sont utilisées dans de nombreux systèmes embarqués :

- **Commande de l'éclairage automatique d'un bâtiment ;**
- **Réglage de la vitesse d'un moteur ou de la luminosité d'un éclairage par PWM ;**
- **Lecture de la position d'un potentiomètre ou d'un joystick ;**
- **Détection de l'appui sur un bouton de commande ;**
- **Acquisition de mesures issues de capteurs ;**
- **Interfaces utilisateur des objets connectés (IoT), robots, drones ou systèmes industriels.**



2 Matériel nécessaire

2.1 Cartes de développement

2.1.1 Carte Arduino Uno

La carte Arduino Uno constitue la plateforme d'apprentissage utilisée lors de la première phase de la formation. Elle permet de découvrir les principes fondamentaux de la programmation embarquée : acquisition d'informations issues de capteurs, traitement des données et commande d'actionneurs.

Caractéristiques principales :

Microcontrôleur : ATmega328P

Fréquence d'horloge : 16 MHz

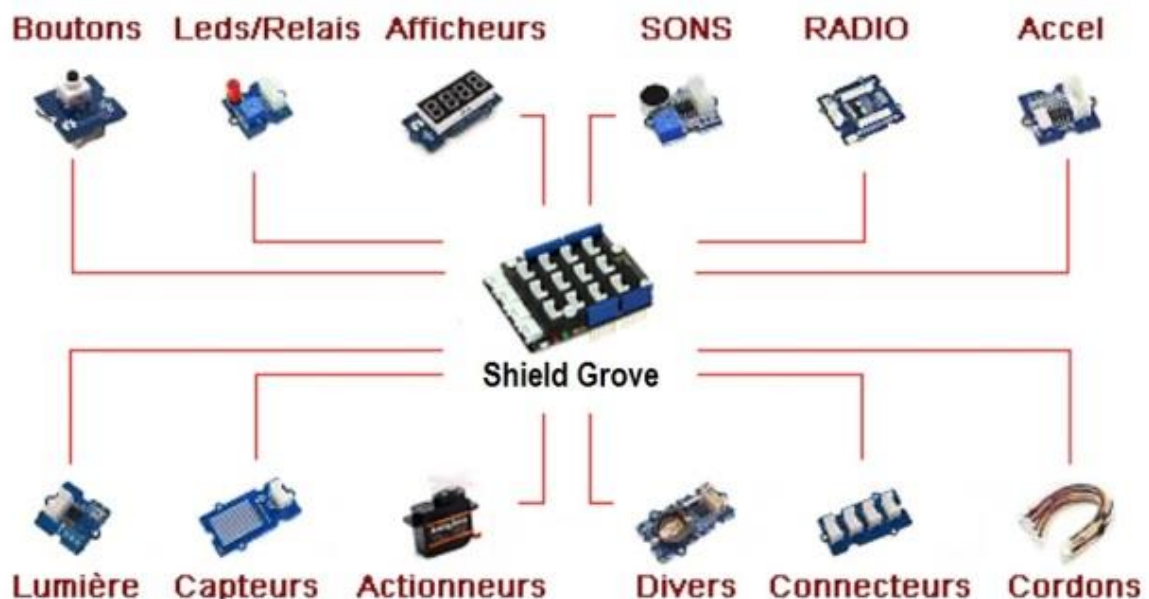
Entrées/sorties numériques : 14 (dont 6 sorties PWM)

Entrées analogiques : 6

Communication : UART, SPI, I²C

Tension de fonctionnement : 5 V

2.1.2 Extensions Grove



Afin de simplifier l'interconnexion entre la carte Arduino et les différents périphériques, des modules Grove sont utilisés.

Le système Grove permet :

- De limiter les erreurs de câblage grâce à des connecteurs normalisés ;
- De faciliter l'expérimentation et le prototypage rapide ;
- De se concentrer sur la programmation et l'exploitation des composants.

Les modules utilisés, tout au long de la formation, permettent d'aborder différentes fonctions d'un système embarqué :

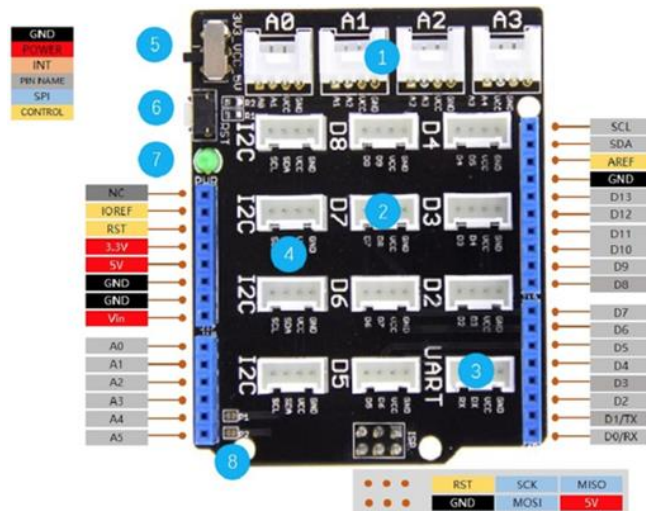
- Capteurs
 - Capteur de température DS18B20
 - Capteur température/humidité DHT11 ou DHT22
 - Potentiomètre (acquisition analogique)
 - Encodeur incrémental (mesure de position/vitesse)
- Interfaces utilisateur
 - Boutons poussoirs
 - Afficheur LCD RGB Grove
- Actionneurs
 - LEDs
 - Buzzer (selon disponibilité)

2.1.3 Carte d'interface Grove

Une carte d'extension Grove Base Shield est associée à la carte Arduino Uno.

Elle permet :

- De connecter directement les modules Grove ;
- D'identifier facilement les entrées/sorties utilisées ;
- De faciliter l'évolution des montages lors des travaux pratiques.



2.1.4 Équipements complémentaires

- Câble USB Arduino pour la programmation et l'alimentation
- Ordinateur équipé de l'environnement de développement
- Câbles de connexion

3 Installation, câblage et configuration de l'environnement Arduino

3.1 Installation

L'IDE Arduino 2.x est normalement déjà installé sur votre poste de travail.

Si ce n'est pas le cas, procédez à son installation en suivant la procédure disponible à l'adresse suivante :

<https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE>

Cet environnement de développement permettra de :

- Créer et modifier des programmes Arduino (sketchs) ;
- Compiler le code source ;
- Téléverser les programmes dans la carte Arduino Uno ;
- Utiliser les bibliothèques nécessaires à l'exploitation des modules Grove.

La version Arduino IDE 2.x est recommandée pour bénéficier des dernières fonctionnalités et améliorations de l'environnement de développement. L'IDE Arduino ainsi que les bibliothèques utilisées doivent être régulièrement mis à jour afin d'éviter d'éventuels problèmes de compilation, de compatibilité ou de fonctionnement lors du développement des applications embarquées.

4 Programmation.

Partie 1 durée conseillée : 1h

4.1 Commander sur une sortie TOR (Tout Ou Rien).

Premier programme : Blink

Dans une première phase d'apprentissage, il est essentiel de s'appuyer sur les nombreux exemples disponibles dans l'IDE Arduino. Ces programmes constituent une base d'expérimentation permettant de découvrir progressivement le fonctionnement des composants, les interactions entre le matériel et le logiciel, ainsi que les principales fonctions de programmation utilisées dans les systèmes embarqués.

L'objectif est d'acquérir de bonnes pratiques de développement, aussi bien dans l'utilisation du matériel que dans l'organisation du code. Une attention particulière devra être portée à la lisibilité et à la structuration des programmes afin de faciliter leur compréhension, leur évolution et leur maintenance, que ce soit lors d'une relecture personnelle ou par un autre développeur.

Objectifs :

Comprendre la structure d'un programme Arduino.

Utiliser `pinMode`(quelle broche ?, quelle orientation ?);

Exemple `pinMode(13, OUTPUT);`

Utiliser `digitalWrite`(quelle broche ?, quel état ?);

Exemple `digitalWrite(LED_BUILTIN, HIGH);`

Modifier la fréquence de clignotement.

`delay`(entier temps en ms);

Exemple `delay(1000);`

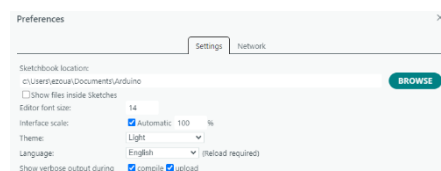
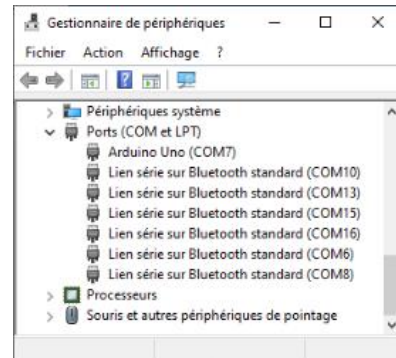
Notions abordées :

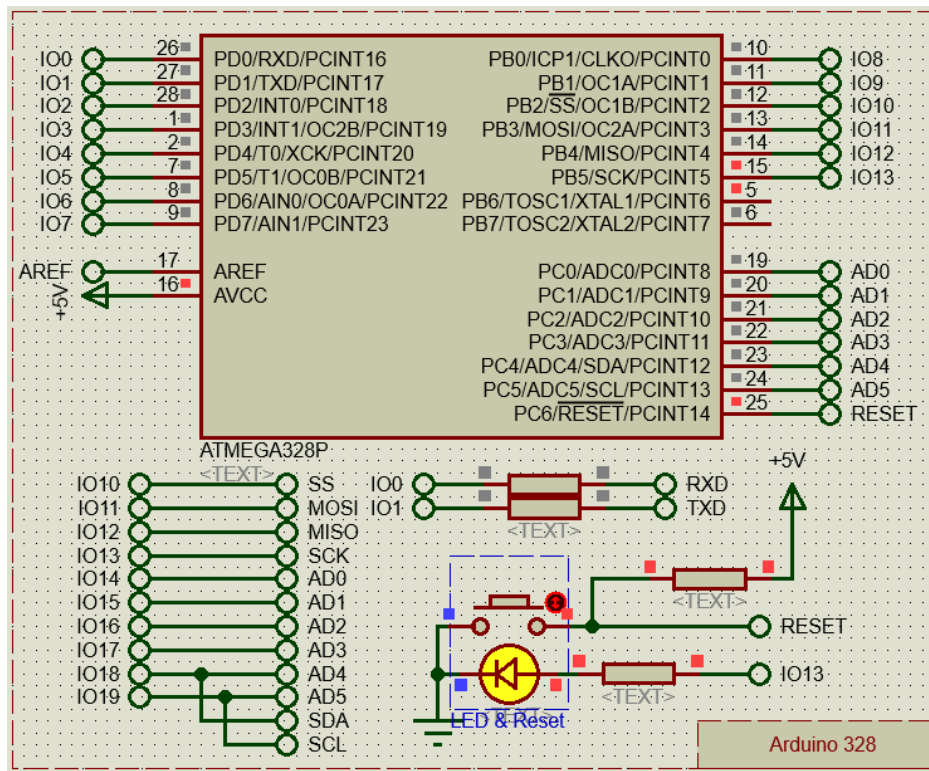
```
void setup() {}
```

```
void loop() {}
```

Avant de modifier le programme, il est important d'en comprendre le fonctionnement. Le tableau ci-dessous associe le code source de l'exemple **Blink** à une représentation de son comportement sous **Proteus**. Cette mise en parallèle permet d'établir le lien entre les instructions du programme et leur effet sur le matériel.

- 1- Lire le tableau de l'API arduino en annexe puis compléter le programme page suivante dans l'IDE Arduino.
- 2- Identifier les deux fonctions principales :
 - a. `setup()`
 - b. `loop()`
- 3- Connecter la carte Arduino Uno à l'ordinateur à l'aide du câble USB.
- 4- Sélectionner la carte de développement utilisée ainsi que le port de communication correspondant à la carte. (Vous pouvez avoir accès au port de la carte grâce au gestionnaire de périphérique rubrique port com)
- 5- Compiler le programme en cliquant sur **Verify** (✓) et vérifier qu'aucune erreur n'est signalée.
- 6- Téléverser le programme dans le microcontrôleur en cliquant sur **Upload** (→).
- 7- Observer le fonctionnement de la carte Arduino et vérifier que la LED intégrée clignote périodiquement.
- 8- Remplacer la constante `LED_BUILTIN` par la valeur `13`, puis recompiler et téléverser le programme. En déduire sur quelle broche de l'Arduino est connectée la LED de la carte Arduino Uno.
- 9- Ouvrir les préférences de l'IDE (File → Preferences) et activer les options permettant d'afficher les informations détaillées de compilation et de téléversement (**Compilation et Upload**).
Ces informations seront utiles tout au long des travaux pratiques pour identifier et corriger les éventuelles erreurs de programmation.
- 10- Recompiler le programme et observer les messages affichés dans la fenêtre **Output**. Identifier notamment :
 - a. Le déroulement des différentes étapes de compilation ;
 - b. La taille du programme (mémoire Flash) ;
 - c. L'utilisation de la mémoire SRAM ;
 - d. Le message confirmant le succès du téléversement.
 - e. Identifier, dans les messages de compilation, le chemin d'accès au fichier compilé généré sur votre ordinateur. Cette information sera utile pour localiser les fichiers produits par la chaîne de compilation et mieux comprendre le processus de génération d'un programme embarqué.





Programme à compléter

```
//faire clignoter la LED intégrée
// La fonction setup() est exécutée UNE SEULE FOIS
// au démarrage ou après un RESET de la carte.
void setup() {
    // LED_BUILTIN correspond à la LED intégrée de l'Arduino UNO.
    // On doit configurer cette broche comme une _____.
    pinMode(LED_BUILTIN, _____);
}
// La fonction loop() est exécutée en boucle indéfiniment.
void loop() {
    // Allumer la LED :
    // HIGH place la sortie à l'état logique 1.
    digitalWrite(LED_BUILTIN, _____);
    // Attendre 1 seconde.
    // delay() attend un nombre de millisecondes.
    // 1000 ms = 1 seconde.
    delay(_____);
    // Éteindre la LED :
    // LOW place la sortie à l'état logique 0.
    digitalWrite(LED_BUILTIN, _____);
    // Attendre à nouveau 1 seconde.
    delay(_____);
}
```



Synthèse :

Questionnaire :

À partir de l'étude du programme **Blink**, répondre aux questions suivantes :

- 1- Quel est le rôle des fonctions `setup()` et `loop()` ?
- 2- Combien de fois chacune de ces fonctions est-elle exécutée lors du fonctionnement du programme ? Justifier votre réponse.
- 3- Pourquoi la fonction `loop()` est-elle indispensable dans un programme Arduino ?
- 4- Les fonctions `setup()` et `loop()` appartiennent-elles à l'API Arduino ? Justifier votre réponse.
- 5- Identifier les fonctions de l'API Arduino utilisées dans le programme ****Blink**** et préciser le rôle de chacune d'elles.
- 6- Pourquoi est-il préférable d'utiliser la constante `LED\_BUILTIN` plutôt que la valeur numérique `13` ?

Simulation du programme avec Proteus (Facultatif) :



Le logiciel Proteus permet de simuler le fonctionnement d'un système embarqué avant sa mise en œuvre sur le matériel réel. Il exécute directement le fichier compilé généré par l'IDE Arduino (.hex) et reproduit le comportement du microcontrôleur ainsi que des composants électroniques associés.

La simulation constitue une solution particulièrement intéressante lorsque le matériel n'est pas disponible. Elle permet également de valider le fonctionnement d'un

programme, de réaliser des essais sans risque pour les composants et de faciliter les opérations de mise au point et de débogage.

Procédure :

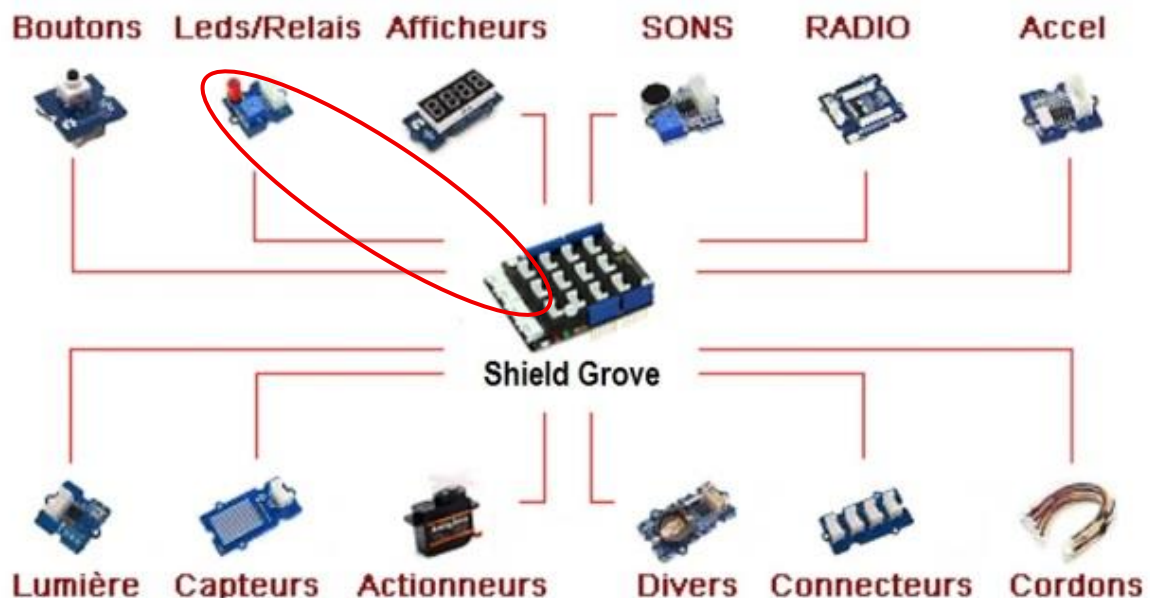
- 1- Compiler le programme **Blink** avec l'IDE Arduino.
- 2- Dans la fenêtre **Output (repéré 10)**, repérer puis copier le chemin d'accès au fichier compilé (Blink.ino.hex), par exemple :
C:\Users\ezoua\AppData\Local\arduino\sketches\35E128331788E7A1049A11850B6BBB12\Blink.ino.hex
- 3- Ouvrir le logiciel **Proteus**.
- 4- Ouvrir le projet **Arduino_328_Blink.pdsprj** fourni.
- 5- Double-cliquer sur le composant **ATMEGA328P** afin d'accéder à ses propriétés.
- 6- Dans le champ **Program File**, coller le chemin d'accès du fichier **.hex** précédemment copié.
- 7- Vérifier que la fréquence du microcontrôleur est configurée à **16 MHz**.
- 8- Cliquer sur OK.
- 9- Cliquer sur le bouton **Play** (en bas à gauche) afin de lancer la simulation.
- 10- Vérifier que la LED clignote de la même manière que sur la carte Arduino réelle.

Questionnaire :

Pourquoi Proteus utilise-t-il le fichier compilé (.hex) et non le fichier source (.ino) ?

Utilisation du matériel Grove :

Attention le matériel est fragile demander de l'aide si nécessaire.



- 1- Débrancher le câble USB du PC.
- 2- Connecter la carte Grove interface appelée Base Shield sur votre carte Arduino.

- 3- Connecter une des cartes grove LED à votre disposition sur la broche 4.
- Modifier votre programme et tester le.
- 4- Rebrancher le câble USB au PC.
- 5- Constater le bon fonctionnement.

4.2 Exercice : faire clignoter une LED 5 fois

La boucle for :

Une LED est connectée sur la broche 8 de la carte Arduino Uno.

Compléter le programme afin que la LED clignote **5 fois**.

La LED doit :

- être allumée pendant 500 ms ;
- être éteinte pendant 500 ms.

Programme à compléter

```
// Broche de connexion de la LED
const int LED = ____;

void setup()
{
    // Configuration de la broche de la LED
    pinMode(LED, OUTPUT);
}

void loop()
{
    // Répéter les instructions 5 fois
    for (int compteur = 0; compteur < ____; ____++)
    {
        // Allumer la LED
        digitalWrite(LED, ____);

        // Attendre 500 ms
        delay(____);

        // Éteindre la LED
        ____ (LED, ____);

        // Attendre 500 ms
        ____ (____);
    }
}
```

4.2.1 Travail demandé

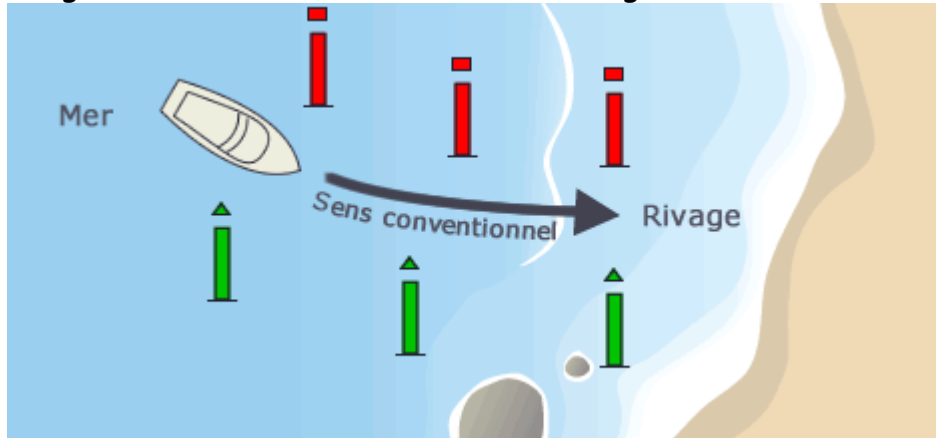
Compléter les éléments manquants :

- le nombre de répétitions de la boucle for ;
- les valeurs HIGH et LOW ;
- les durées des delay().

Question : combien de fois la LED doit-elle clignoter ?

Mission – À vous de jouer !

Contexte : **Programmation d'une balise maritime de signalisation**



Une société spécialisée dans les équipements maritimes développe une nouvelle génération de balises lumineuses autonomes destinées à sécuriser l'entrée des ports. Lors de sa mise sous tension, la balise doit réaliser une séquence de test permettant de vérifier le bon fonctionnement des feux lumineux. Une fois cette phase terminée, elle passe en mode normal de fonctionnement afin d'être visible par les navigateurs. La maquette de la balise est réalisée à l'aide d'une carte Arduino Uno équipée de deux LEDs représentant les feux de signalisation :

- une LED verte indiquant une zone sécurisée ;
- une LED rouge indiquant une zone dangereuse.

Votre mission consiste à programmer le fonctionnement de cette balise conformément au cahier des charges suivant.

Cahier des charges

1- Phase d'initialisation – Mise en service de la balise

Lors de chaque mise sous tension, la balise doit effectuer une séquence de test :

- Les deux LEDs doivent clignoter rapidement **5 fois** afin de signaler le démarrage du système.
- La durée d'allumage et d'extinction de chaque LED est fixée à **500 ms**.

À l'issue de cette phase, la balise entre automatiquement en fonctionnement normal.

2- Mode normal de fonctionnement

En fonctionnement normal, la balise doit reproduire un cycle lumineux permanent :

Phase 1 : signalisation rapide

- La LED rouge doit clignoter **10 fois**.
- La période de clignotement est de **500 ms** (250 ms allumée / 250 ms éteinte).

Phase 2 : signalisation lente

- La LED verte doit clignoter **5 fois**.
- La période de clignotement est de **2 secondes** (1 seconde allumée / 1 seconde éteinte).

À la fin de ces deux phases, le cycle recommence indéfiniment.

3- Contraintes de programmation

Le programme devra respecter les contraintes suivantes :

- Utiliser les fonctions setup() et loop() de l'environnement Arduino.
- Utiliser les fonctions de l'API Arduino nécessaires au pilotage des LEDs :
 - pinMode()
 - digitalWrite()
 - delay()
- Utiliser une boucle for afin de réaliser les répétitions de clignotement.
- Organiser le programme afin de rendre le code lisible et compréhensible :
 - Utiliser des commentaires ;
 - Utiliser des noms de variables explicites ;
 - Respecter une indentation correcte.
- Tester le fonctionnement du programme sur la carte Arduino Uno.

Partie 2 durée conseillée : 45 min

4.3 Lecture d'un bouton poussoir

Objectifs :

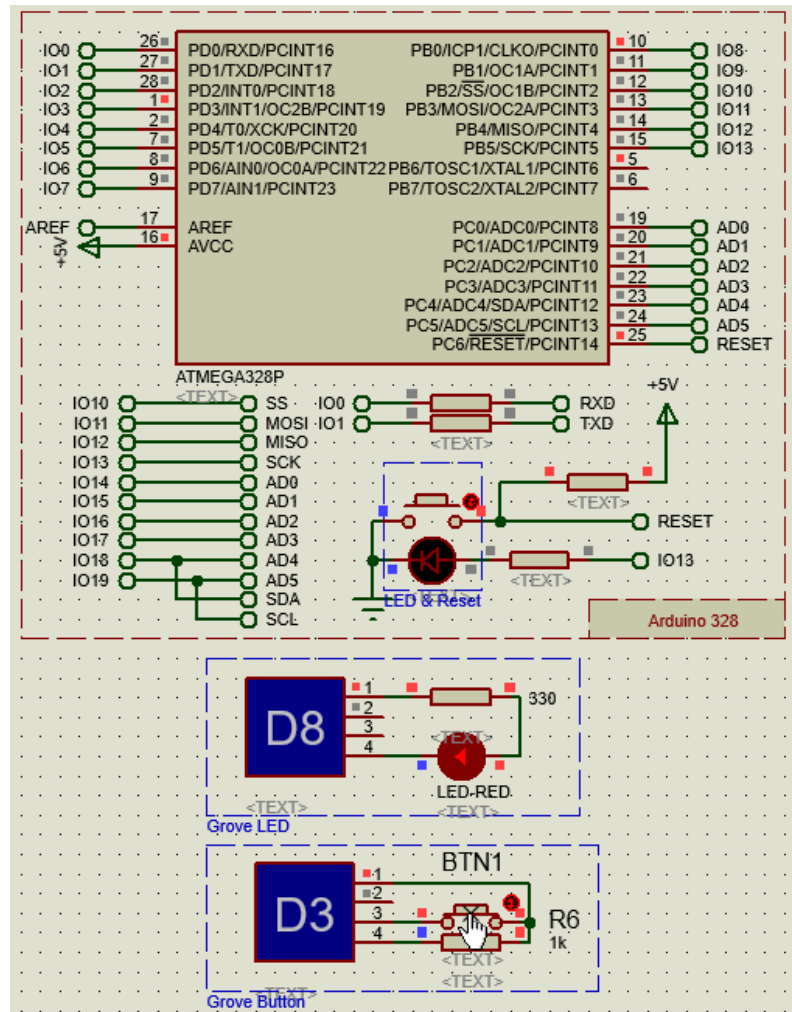
Configurer une entrée numérique.

Lire l'état d'un bouton.

Utiliser une résistance Pull-Up interne.

API :

```
pinMode(quelle broche ?, INPUT ou INPUT_PULLUP);  
Exemple      pinMode(buttonPin, INPUT_PULLUP);  
digitalRead(quelle broche ?);      Exemple : digitalRead(buttonPin);
```



Programme à compléter

```
//Commande d'une LED par un bouton-poussoir
// Broche utilisée pour le bouton-poussoir
// Le bouton est connecté sur la broche numérique 3.
const int buttonPin = ____;
// Broche utilisée pour la LED
const int ledPin = ____;
// Variable permettant de mémoriser l'état du bouton
int buttonState = ____;
void setup() {
  // La LED est un composant de SORTIE.
  pinMode(ledPin, ____);
  // Le bouton est configuré en entrée avec
  // la résistance de rappel interne de l'Arduino.
  pinMode(buttonPin, ____);}
void loop() {
  // Lire l'état logique du bouton et
  // mémoriser le résultat dans buttonState.
  buttonState = ____ (buttonPin);
  // Tester l'état du bouton.
```

```
// Avec INPUT_PULLUP :
// HIGH = bouton relâché
// LOW = bouton appuyé
if (buttonState == _____) {
    // Le bouton est relâché :
    // allumer la LED.
    digitalWrite(ledPin, _____);
} else {
    // Le bouton est appuyé :
    // éteindre la LED.
    digitalWrite(ledPin, _____);
}
}
```

Travail demandé

1. Réaliser le câblage du bouton poussoir conformément au schéma structurel fourni.
2. Compléter le programme dans l'IDE Arduino.
3. Compiler et téléverser le programme d'exemple sur la carte Arduino afin d'en vérifier le fonctionnement.
4. Adapter le programme afin qu'il lise l'état du bouton poussoir connecté sur l'entrée **D3** et commande la LED connectée sur la sortie **D8**, conformément au schéma structurel.
5. Compiler puis téléverser le programme modifié.
6. Décrire le comportement du système obtenu.
7. Optimiser le programme de manière à ce que la fonction loop() ne contienne qu'une seule instruction, sans utiliser de variable intermédiaire. Exploiter pour cela un appel imbriqué des fonction digitalWrite() et digitalRead().

Questionnaire de synthèse

1. Quel est le rôle de la fonction digitalRead() ?
2. Quel est le rôle de la fonction digitalWrite() ?
3. Quel type de valeur est renvoyé par digitalRead() ?
4. À quoi correspondent les constantes HIGH et LOW ?
5. Pourquoi est-il nécessaire de configurer les broches dans la fonction setup() ?
6. Que se passe-t-il lorsque le bouton est appuyé ? Justifier votre réponse.
7. Quel est l'intérêt d'utiliser un programme d'exemple fourni par l'IDE Arduino plutôt que de développer directement son propre programme ?
8. Expliquer pourquoi l'instruction suivante fonctionne :
digitalWrite(8, digitalRead(2));
9. Cette écriture est-elle plus performante que l'utilisation d'une variable intermédiaire ? Justifier votre réponse.
10. Quelle est la différence entre une **entrée numérique** et une **sortie numérique** ?

Si vous ne disposez pas de la carte Arduino ou si vous souhaitez valider votre programme avant le téléversement, vous pouvez le tester à l'aide du projet Proteus **Arduino_328_Button.pdsprj**.

Comme dans le TP précédent, compilez votre programme avec l'IDE Arduino, récupérez le fichier compilé (.hex), puis chargez-le dans le microcontrôleur **ATmega328P** du projet Proteus avant de lancer la simulation.

4.4 Utilisation du Moniteur Série

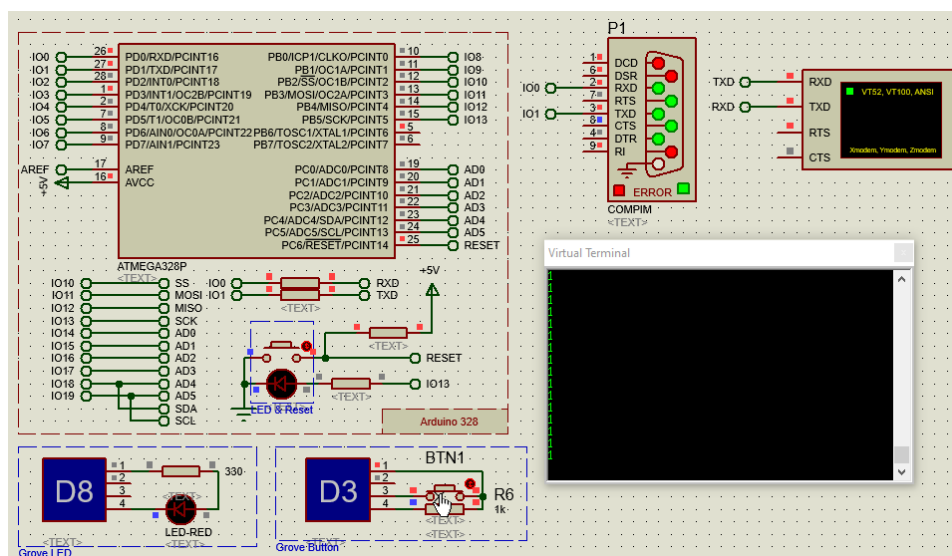
Objectifs :

- Initialiser la liaison série.
- Afficher des messages.
- Déboguer un programme.

API :

- `Serial.begin(vitesse de transmission);` Exemple : `Serial.begin(9600);`
- `Serial.print(« Texte »);` Exemple : `Serial.print("Le bouton est :")`
- `Serial.println(NomDuneVariable);` Exemple : `Serial.println(buttonState);`

Schéma structurel



Programme à compléter

```
// lecture d'un bouton-poussoir
// Affichage de l'état du bouton sur le moniteur série
// Broche numérique utilisée pour le bouton
int pushButton = ____;
void setup() {
    // Initialise la communication série à 9600 bauds
    Serial.begin(______);
    // Configure la broche du bouton comme une entrée
    pinMode(pushButton, ____);
}
void loop() {

    // Lit l'état logique de la broche du bouton
```

```
// digitalWrite() retourne HIGH ou LOW
```

AVIGNON
UNIVERSITÉ

```
int buttonState = _____;
// Affiche l'état du bouton sur le moniteur série
Serial._____ (buttonState);
// Attendre 1 ms avant la prochaine lecture
delay(_____);
}
```

Travail demandé

Partie 1 - Découverte du Moniteur Série

1. Compléter le programme dans l'IDE Arduino.

2. Identifier les fonctions de l'API Arduino utilisées pour initialiser et exploiter la liaison série.

3. Adapter le programme pour effectuer la lecture de la broche 3.

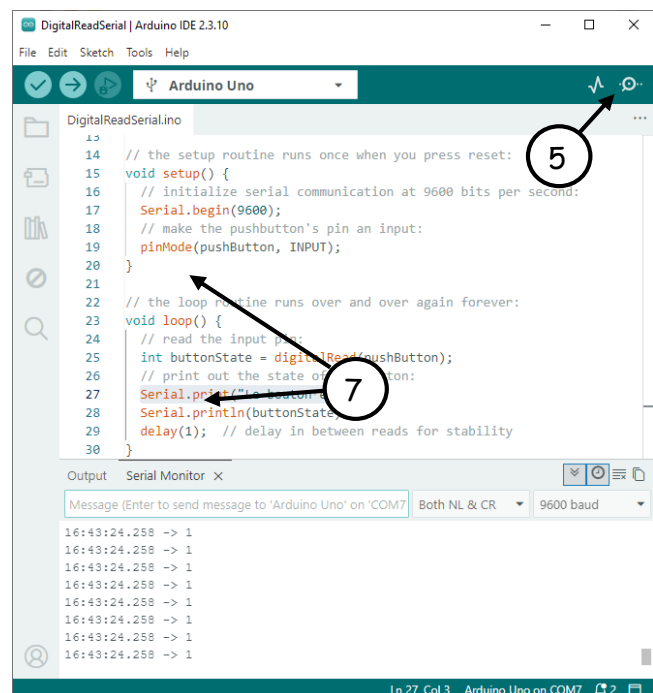
4. Compiler puis téléverser le programme sur la carte Arduino Uno.

5. Ouvrir le **Moniteur Série**.

6. Vérifier que la vitesse de communication (baudrate) du Moniteur Série correspond à celle définie dans le programme.

7. Observer les informations affichées et décrire le comportement du programme.

Questionnaire de synthèse



1. Quel est le rôle de la fonction Serial.begin() ?

2. Que représente la valeur transmise à Serial.begin() (par exemple 9600 ou 115200) ?

3. Quelle est la différence entre Serial.print() et Serial.println() ?

4. Pourquoi faut-il ouvrir le Moniteur Série après avoir téléversé le programme ?

5. Que se passe-t-il si la vitesse sélectionnée dans le Moniteur Série est différente de celle définie dans le programme ?

6. À quel moment la fonction Serial.begin() doit-elle être appelée ? Justifier votre réponse.

7. Pourquoi est-il préférable de placer Serial.begin() dans la fonction setup() ?

8. Pourquoi l'affichage de nombreuses informations sur le port série peut-il ralentir l'exécution d'un programme embarqué ?

Si vous ne disposez pas de la carte Arduino ou si vous souhaitez valider votre programme avant le téléversement, vous pouvez le tester à l'aide du projet Proteus **Arduino 328_DigitalReadSerial.pdsprj**.

Comme dans le TP précédent, compilez votre programme avec l'IDE Arduino, récupérez le fichier compilé (.hex), puis chargez-le dans le microcontrôleur **ATmega328P** du projet Proteus avant de lancer la simulation.

Partie 3 durée conseillée : 1h

4.5 Acquisition d'une grandeur analogique

Objectifs

- Découvrir les entrées analogiques de la carte Arduino Uno.
- Comprendre le rôle du convertisseur Analogique / Numérique (CAN).
- Lire la valeur d'un potentiomètre.
- Afficher une mesure dans le Moniteur Série.
- Exploiter le programme d'exemple **AnalogInput**.

API étudiée

- `analogRead(entréeAnalogique)` Exemple : `analogRead(A0)`

Introduction

De nombreux systèmes embarqués doivent acquérir des grandeurs physiques issues de leur environnement, telles que la température, la luminosité, la pression ou encore la position d'un potentiomètre. Ces grandeurs sont généralement de nature analogique et évoluent de manière continue.

Le microcontrôleur ne pouvant traiter directement une tension analogique, il utilise un **convertisseur Analogique / Numérique (CAN)** qui transforme cette tension en une valeur numérique exploitable par le programme.

Dans ce travail pratique, vous étudierez le fonctionnement d'une entrée analogique en utilisant un potentiomètre connecté à la carte Arduino Uno. La valeur mesurée sera affichée dans le Moniteur Série afin d'observer l'évolution de la conversion numérique.

Exemples de contexte

🌡 **Thermostat** : mesure de la température ambiante à l'aide d'un capteur analogique.

☀ **Station météorologique** : mesure de l'intensité lumineuse grâce à une photorésistance.

🚗 **Automobile** : lecture de la position de la pédale d'accélérateur ou du niveau de carburant.

🚢 **Balise maritime** : mesure de la position d'un potentiomètre permettant de régler l'intensité du feu lumineux.

🤖 **Robotique** : lecture de la position d'un joystick ou d'un potentiomètre de commande.

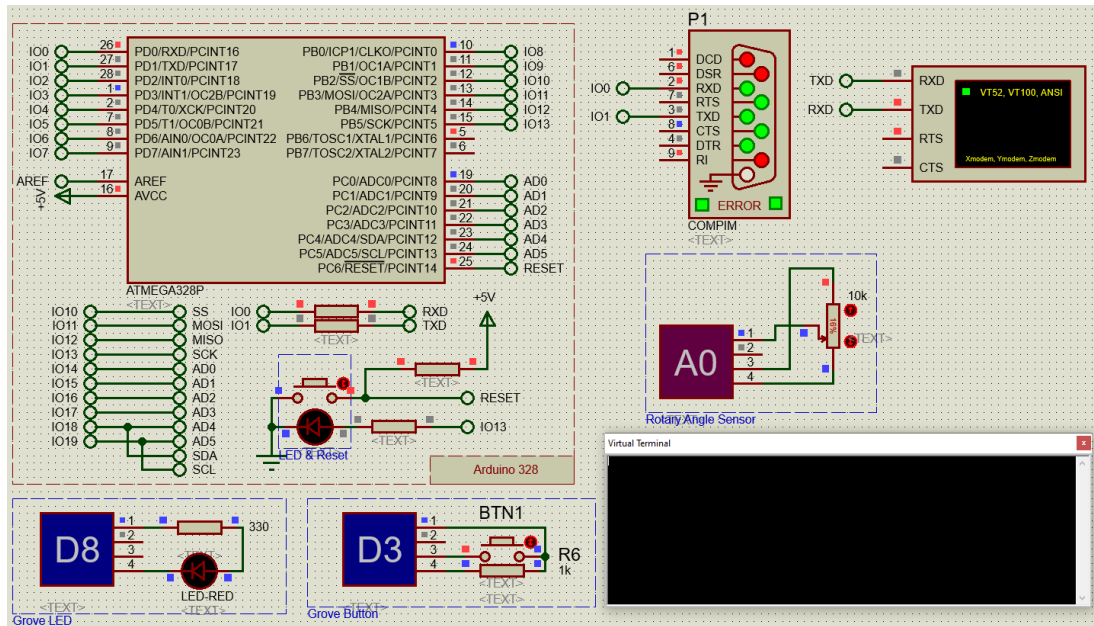
 **Agriculture connectée** : mesure de l'humidité du sol pour piloter un système d'irrigation.

 **Industrie** : acquisition de la position d'une vanne ou d'un capteur de pression analogique.

 **Domotique** : mesure de la luminosité d'une pièce afin d'adapter automatiquement l'éclairage.

 **Instrumentation** : lecture de la position d'un potentiomètre utilisé pour le réglage d'une consigne.

Schéma structurel



Programme à compléter

```
// Potentiomètre → entrée analogique → vitesse de clignotement
// Broche analogique utilisée pour le potentiomètre
int sensorPin = ____;
// Broche numérique utilisée pour la LED
int ledPin = ____;
// Variable permettant de mémoriser la valeur du potentiomètre
int sensorValue = ____;
void setup() {
    // Configure la broche de la LED comme une sortie
    pinMode(ledPin, ____);
}
void loop() {
    // Lire la valeur analogique du potentiomètre
    // analogRead() retourne une valeur comprise entre 0 et 1023
    sensorValue = ____ (sensorPin);
    // Allumer la LED
    digitalWrite(ledPin, ____);
    // Attendre pendant la durée indiquée par le potentiomètre
    // Plus sensorValue est grand, plus la LED reste allumée longtemps
```

```

delay(____);
// Éteindre la LED
digitalWrite(ledPin, _____);
// Attendre la même durée
delay(____);
}

```

Les entrées analogiques de l'Arduino Uno

Contrairement aux entrées/sorties numériques, les mesures analogiques ne peuvent être réalisées que sur les broches **A0 à A5** de la carte Arduino Uno.

Cette limitation est liée au matériel : le microcontrôleur **ATmega328P** intègre un **convertisseur Analogique / Numérique (CAN)** de **10 bits**, accessible uniquement sur ces six entrées analogiques. Les broches numériques (D0 à D13) ne sont pas reliées à ce convertisseur et ne peuvent donc pas être utilisées avec la fonction `analogRead()`.

Les six entrées analogiques permettent de mesurer une tension comprise entre **0 V et 5 V**, qui est convertie en une valeur numérique comprise entre **0 et 1023**.

Fonction	Broches utilisables	Valeur retournée
<code>digitalRead()</code>	D0 à D13 et A0 à A5	HIGH ou LOW
<code>analogRead()</code>	A0 à A5 uniquement	0 à 1023

Cette courbe illustre la conversion analogique-numérique réalisée par le CAN 10 bits : la tension d'entrée V_{in} , comprise entre 0 et V_{Ref} , est transformée en une valeur numérique N comprise entre 0 et 1023 (soit $2^{10}-1$). Chaque niveau numérique correspond à un intervalle de tension appelé quantum de conversion.

$$N = \frac{V_{entrée}}{V_{référence}} \times 1023$$

Avec :

- $V_{entrée}$: tension analogique mesurée par l'entrée ADC
- $V_{référence}$: tension de référence du convertisseur
- N : résultat numérique fourni par le CAN

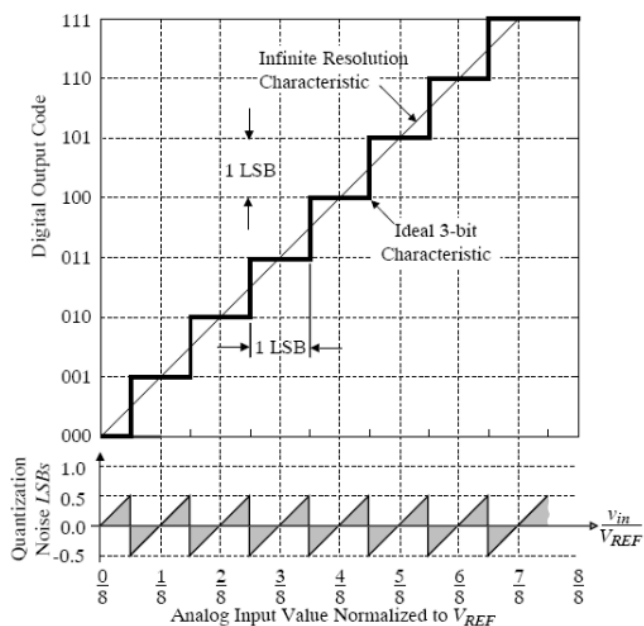


Figure 10.5-3 Ideal input-output characteristics of a 3-bit ADC.

Travail demandé

- 1- Ajouter le potentiomètre en connectant la carte Rotary Angle Sensor à votre disposition sur A0 de la carte de connexions Grove.

- 2- Compléter le programme dans l'IDE Arduino.
- 3- Compiler et Téléverser votre programme.
- 4- Décrire le comportement de votre programme.
- 5- Compléter le programme pour renvoyer le résultat de la conversion sur la liaison série.

Questionnaire de synthèse

Compréhension du fonctionnement

1. Quel est le rôle d'un convertisseur analogique-numérique (CAN) ?
2. Pourquoi un potentiomètre peut-il être utilisé comme capteur analogique ?
3. Quelle est la résolution du CAN intégré dans l'ATmega328P ?
 - ☐ 8 bits
 - ☐ 10 bits
 - ☐ 12 bits
 - ☐ 16 bits
4. Combien de niveaux différents peut fournir un CAN 10 bits ?
5. Quelles sont les valeurs minimale et maximale retournées par la fonction `analogRead()` ?
Minimum :
Maximum :

Programmation Arduino

6. Quelle fonction Arduino permet de lire une entrée analogique ?
7. Quelle fonction permet de configurer une broche comme sortie numérique ?
8. Quelle fonction permet de modifier l'état logique d'une sortie numérique ?
9. Pourquoi utilise-t-on une variable de type `int` pour stocker le résultat de `analogRead()` ?

Analyse système

10. Compléter la chaîne d'information du système :
Potentiomètre → → Microcontrôleur → → LED
11. La valeur retournée par le CAN est-elle une tension ? Justifier.
12. Si la tension mesurée vaut la moitié de la tension de référence, quelle valeur numérique doit-on obtenir environ ?

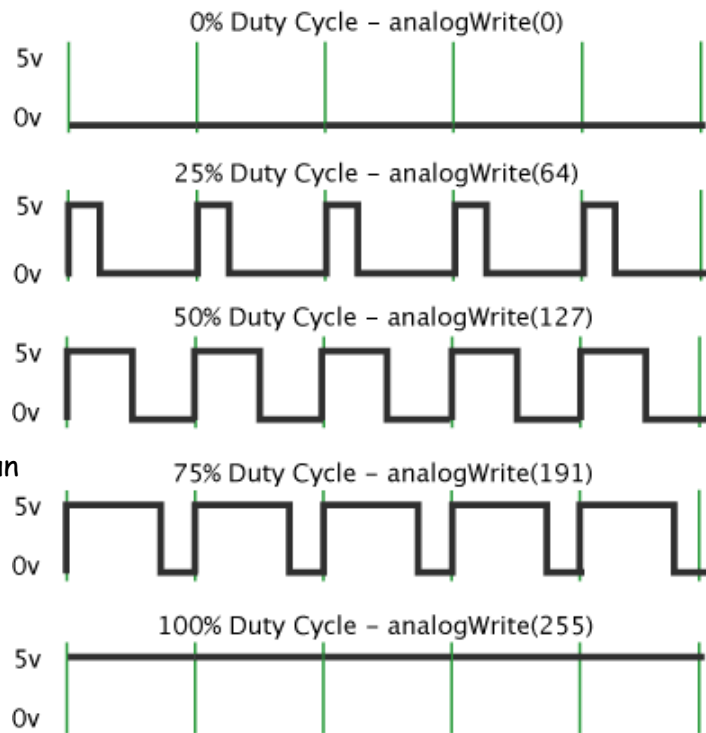
Dans de nombreuses applications embarquées, il est nécessaire de faire varier la puissance fournie à un actionneur sans disposer d'une véritable sortie analogique. C'est notamment le cas pour régler la luminosité d'une LED, la vitesse d'un moteur à courant continu, l'intensité d'un éclairage, la position d'un servomoteur ou encore le volume d'un buzzer.

Pour répondre à ce besoin, les microcontrôleurs utilisent une technique appelée **PWM** (*Pulse Width Modulation*), ou **modulation de largeur**

d'impulsion (MLI). Le principe consiste à générer un signal numérique qui alterne rapidement entre les états **HIGH** et **LOW**. En faisant varier la durée pendant laquelle le signal reste à l'état HIGH (rapport cyclique), il est possible de modifier la puissance moyenne transmise à l'actionneur.

Bien que la sortie du microcontrôleur reste uniquement numérique (0 V ou 5 V), l'effet obtenu est perçu comme une variation continue de la puissance. Par exemple, une LED semblera plus ou moins lumineuse, tandis qu'un moteur tournera plus ou moins vite.

Dans ce travail pratique, vous découvrirez le fonctionnement d'une sortie PWM et apprendrez à contrôler progressivement la luminosité d'une LED à l'aide de la fonction `analogWrite()`.



Exemples de contexte

-  **Éclairage public** : variation progressive de l'intensité des lampadaires.
-  **Automobile** : réglage de la luminosité du tableau de bord.
-  **Balise maritime** : modulation de l'intensité d'un feu de signalisation pour améliorer sa visibilité.
-  **Robotique** : commande de la vitesse d'un moteur à courant continu.
-  **Électronique grand public** : réglage de la luminosité d'un écran ou du rétroéclairage d'un clavier.

Objectifs

Acquisition analogique → traitement → sortie PWM → affichage sur le Moniteur Série

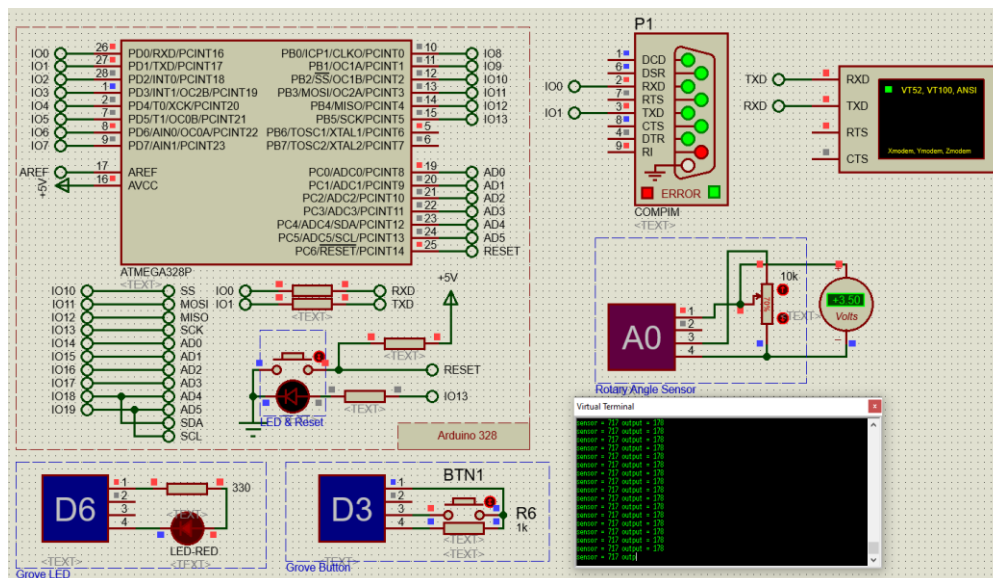
- Lire une grandeur analogique à l'aide de `analogRead()`.
- Comprendre le rôle du **CAN 10 bits** de l'ATmega328P.
- Convertir une mesure analogique comprise entre **0 et 1023**.

- Adapter une valeur numérique avec la fonction `map()`.
- Commander une sortie PWM avec `analogWrite()`.
- Afficher des informations dans le Moniteur Série.
- Mettre en relation une **entrée**, un **traitement** et une **sortie**.

API

- `map(sensorValue, 0, 1023, 0, 255);`
- `analogWrite(analogOutPin, outputValue);`

Schéma structurel



Programme à compiler

```
// Potentiomètre → entrée analogique → sortie PWM
// Broche analogique du potentiomètre
const int analogInPin = ____;
// Broche PWM utilisée pour commander la LED
const int analogOutPin = ____;
// Valeur lue sur l'entrée analogique
int sensorValue = ____;
// Valeur convertie pour la sortie PWM
int outputValue = ____;
void setup() {
    // Initialiser la communication série à 9600 bauds
    Serial._____(____);
}
void loop() {
    // Lire la valeur du potentiomètre
    // Arduino UNO : valeur comprise entre 0 et 1023
    sensorValue = _____(analogInPin);
    // Convertir la valeur 0-1023 en valeur 0-255
    // afin de pouvoir commander la sortie PWM.
    outputValue = _____(
        sensorValue,
```

```

    0, 1023,
    0, 255
);
// Générer un signal PWM sur la sortie
// outputValue détermine le rapport cyclique.
analogWrite(analogOutPin, outputValue);
// Afficher la valeur du potentiomètre
Serial.print("sensor = ");
Serial.print(sensorValue);
// Afficher la valeur PWM
Serial.print("\t output = ");
Serial.print(outputValue);
// Attendre 2 ms
delay(2);
}

```

Travail demandé

- 1- Compléter le programme dans l'IDE Arduino.
- 2- Lire attentivement le programme et identifier :
 - la broche d'entrée analogique ;
 - la broche de sortie PWM ;
 - les variables utilisées pour stocker les mesures ;
 - les fonctions analogRead(), map() et analogWrite() ;
 - les fonctions utilisées pour communiquer avec le Moniteur Série.
- 3- Adapter le programme aux entrées et sorties définies dans le schéma structurel.
- 4- Effectuer le câblage des périphériques d'entrées et sorties.
- 5- Compiler puis téléverser le programme.
- 6- Ouvrir le **Moniteur Série** et régler la vitesse à **9600 bauds**.
- 7- Faire varier progressivement la position du potentiomètre.
- 8- Observer simultanément :
 - La valeur sensor affichée dans le Moniteur Série ;
 - La valeur output ;
 - La luminosité de la LED.
- 9- Décrire l'évolution de chacune de ces grandeurs.
- 10- Expliquer le rôle de l'instruction :


```
outputValue = map(sensorValue, 0, 1023, 0, 255);
```
- 11- Expliquer pourquoi la valeur obtenue par analogRead() est comprise entre **0 et 1023**, alors que la valeur utilisée par analogWrite() est comprise entre **0 et 255**.
- 12- Modifier le programme afin d'afficher également la tension correspondant à la valeur mesurée par le CAN.

Questionnaire de synthèse

Compréhension du système

1. Quel est le rôle du potentiomètre dans ce système ?
2. Quel composant du microcontrôleur réalise la conversion analogique-numérique ?
3. Quelle est la résolution du CAN de l'ATmega328P ?

4. Quelles sont les valeurs minimale et maximale retournées par `analogRead()` ?

5. Combien de niveaux différents peuvent être représentés avec un CAN 10 bits ?

Analyse du programme

6. Quel est le rôle de l'instruction : `sensorValue = analogRead(analogInPin);`

7. Quel est le rôle de la fonction `map()` ?

8. Pourquoi faut-il convertir la valeur **0-1023** en **0-255** avant d'utiliser `analogWrite()` ?

9. Quel est le rôle de : `analogWrite(analogOutPin, outputValue);`

10. Que représente la valeur `outputValue` ?

11. Quel est le rôle des fonctions `Serial.print()` et `Serial.println()` ?

12. Pourquoi la valeur affichée par `sensor` évolue-t-elle lorsque l'on tourne le potentiomètre ?

Analyse du système

13. Que se passe-t-il lorsque le potentiomètre est en position minimale ?

14. Que se passe-t-il lorsqu'il est en position maximale ?

15. Pour une tension correspondant à environ la moitié de la tension de référence, quelle valeur devrait retourner approximativement `analogRead()` ?

16. Quelle valeur devrait alors fournir `map()` ?

Simulation (facultatif)

Si vous ne disposez pas de la carte Arduino ou si vous souhaitez valider votre programme avant le téléversement, vous pouvez le tester à l'aide du projet Proteus **Arduino 328_AnalogInOutSerial.pdsprj**.

Comme dans le TP précédent, compilez votre programme avec l'IDE Arduino, récupérez le fichier compilé (.hex), puis chargez-le dans le microcontrôleur **ATmega328P** du projet Proteus avant de lancer la simulation.

5 Activité de synthèse – Commande intelligente d'un feu de signalisation

Contexte

Une société spécialisée dans les équipements de signalisation développe une nouvelle génération de **feux de signalisation à intensité variable**.

Afin d'améliorer la visibilité du feu en fonction des conditions d'utilisation, le système doit permettre à un opérateur de régler progressivement l'intensité lumineuse à l'aide d'un **potentiomètre**.

Le système doit également disposer d'un bouton permettant d'activer ou de désactiver le feu. Les informations de fonctionnement doivent pouvoir être observées à l'aide du **Moniteur Série**.

La maquette est réalisée à partir d'une **carte Arduino Uno**, d'une carte d'interface Grove, d'un potentiomètre, d'un bouton poussoir et d'une LED.

Cahier des charges

1. Mise en service

Lors de la mise sous tension :

- le programme initialise les entrées et sorties nécessaires ;
- la liaison série est initialisée à **9600 bauds** ;
- un message indiquant que le système est opérationnel est affiché dans le Moniteur Série.

2. Commande du feu

Le bouton poussoir permet de commander l'état du feu :

- lorsque le feu est **désactivé**, la LED est éteinte ;
- lorsque le feu est **activé**, la LED est allumée avec une intensité déterminée par la position du potentiomètre.

3. Réglage de l'intensité lumineuse

Le potentiomètre est connecté sur **A0**.

4. Informations dans le Moniteur Série

Lorsque le système fonctionne, le Moniteur Série doit afficher régulièrement :

- l'état du feu ;
- la valeur issue du potentiomètre ;
- la valeur PWM appliquée à la LED.

Exemple :

Feu : ON

Potentiometre : 512

PWM : 127

5. Contraintes de programmation

Le programme devra :

- utiliser `setup()` et `loop()` ;
- utiliser `pinMode()` ;
- utiliser `digitalRead()` ;
- utiliser `digitalWrite()` ;

- utiliser `analogRead()` ;
- utiliser `map()` ;
- utiliser `analogWrite()` ;
- utiliser `Serial.begin()` ;
- utiliser `Serial.print()` et/ou `Serial.println()` ;
- utiliser des variables explicites ;
- apporter des commentaires permettant de comprendre le fonctionnement du programme.

Travail demandé

1. Réaliser le câblage du système conformément au schéma fourni.
2. Identifier les entrées et sorties utilisées.
3. Compléter l'algorithme de fonctionnement du système.
4. Développer le programme à partir des exemples étudiés dans le TP.
5. Compiler le programme et corriger les éventuelles erreurs.
6. Téléverser le programme dans la carte Arduino Uno.
7. Tester successivement :
 - l'activation et la désactivation du feu ;
 - la variation de la position du potentiomètre ;
 - la variation de la luminosité de la LED ;
 - les informations affichées dans le Moniteur Série.
8. Vérifier que la valeur issue du CAN varie bien entre **0 et 1023**.
9. Vérifier que la valeur utilisée pour le PWM varie entre **0 et 255**.
10. Décrire le fonctionnement obtenu.

Questions d'analyse

1. Quel élément du système fournit une information analogique ?
2. Quel composant permet de convertir cette information en valeur numérique ?
3. Pourquoi `analogRead()` retourne-t-elle une valeur comprise entre 0 et 1023 ?
4. Pourquoi utilise-t-on `map()` avant `analogWrite()` ?
5. Quelle est la différence entre **CAN** et **PWM** ?
6. Quel est le rôle du bouton poussoir dans le système ?
7. Quel est le rôle du Moniteur Série ?

6 Conclusion du TP

Au cours de ce TP, vous avez découvert les principes fondamentaux de la programmation embarquée avec une carte Arduino Uno et le microcontrôleur ATmega328P.

Vous avez appris à :

Structurer un programme Arduino avec `setup()` et `loop()` ;

Commander des sorties numériques et lire des entrées numériques ;

Acquérir une grandeur analogique à l'aide du CAN 10 bits et de `analogRead()` ;

Traiter et adapter une valeur numérique avec `map()` ;

Commander progressivement un actionneur à l'aide d'un signal PWM et de `analogWrite()` ;

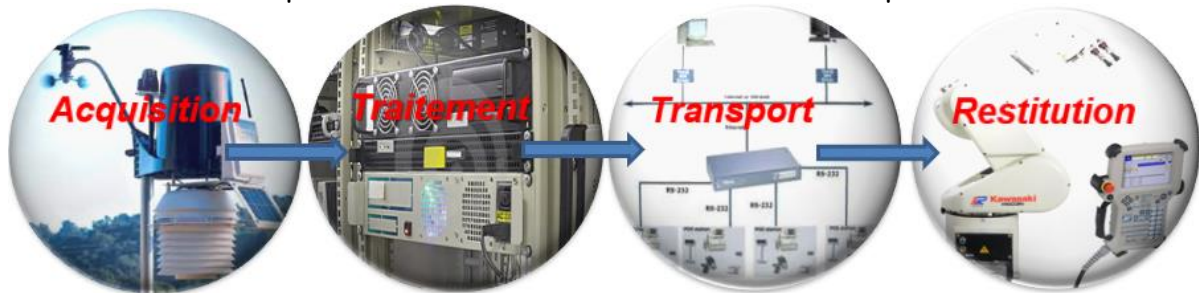
Utiliser le Moniteur Série pour afficher des informations et faciliter le débogage ;

Compiler, téléverser et tester un programme sur une carte Arduino Uno ;

Mettre en relation le programme, le microcontrôleur, les capteurs et les actionneurs.

Ces activités permettent ainsi de comprendre la chaîne élémentaire d'un système embarqué :

Acquérir → Traiter → Commander → Communiquer



Les notions étudiées constituent les bases nécessaires pour aborder des systèmes embarqués plus complexes intégrant des capteurs, des interfaces utilisateur, des actionneurs et, dans la suite de la formation, des systèmes connectés reposant notamment sur l'ESP32.

Tableau de synthèse des entrées et sorties d'un système embarqué

Nom	Capteur / Actionneur	TOR / Analogique	Interface	Fonction Arduino	Application	Rôle
Bouton poussoir	Capteur	TOR	GPIO	<code>digitalRead()</code>	Commande utilisateur	Détecter un appui
Potentiomètre	Capteur	Analogique	CAN / GPIO analogique	<code>analogRead()</code>	Réglage d'une valeur	Mesurer une position ou une consigne
Capteur de température DS18B20	Capteur	Numérique	1-Wire	Bibliothèque <code>DallasTemperature</code>	Thermomètre	Mesurer la température
DHT11 / DHT22	Capteur	Numérique	GPIO	Bibliothèque <code>DHT</code>	Station météo	Mesurer température et humidité
Capteur de luminosité	Capteur	Analogique	CAN	<code>analogRead()</code>	Éclairage automatique	Mesurer la luminosité
Encodeur rotatif	Capteur	TOR	GPIO	<code>digitalRead()</code> / interruption	Réglage, mesure de position	Détecter la rotation et le sens
MPU9250 / Grove 6 axes	Capteur	Numérique	I ² C	Bibliothèque du capteur	Détection de mouvement	Mesurer accélération et rotation
LED	Actionneur	TOR	GPIO	<code>digitalWrite()</code>	Voyant lumineux	Informar l'utilisateur
LED avec variation d'intensité	Actionneur	Analogique simulé / PWM	GPIO PWM	<code>analogWrite()</code> ou PWM ESP32	Variateur lumineux	Modifier l'intensité lumineuse
Buzzer	Actionneur	TOR / PWM	GPIO	<code>digitalWrite()</code> / PWM	Alarme sonore	Produire un signal sonore
Écran LCD RGB Grove	Actionneur / IHM	Numérique	I ² C	Bibliothèque <code>rgb_lcd</code>	Affichage de données	Informar l'utilisateur
Servomoteur	Actionneur	PWM	GPIO PWM	Bibliothèque <code>Servo</code>	Robotique, automatisme	Positionner un mécanisme

Nom	Capteur / Actionneur	TOR / Analogique	Interface	Fonction Arduino	Application	Rôle
Moteur à courant continu	Actionneur	TOR / PWM	GPIO + driver	<code>digitalWrite()</code> / PWM	Robot mobile	Produire un mouvement de rotation

Tableau de synthèse — API Arduino

API / fonction	Rôle	Syntaxe générique	Exemple
<code>pinMode()</code>	Configurer le mode d'une broche	<code>pinMode(broche, mode);</code>	<code>pinMode(35, INPUT);</code>
<code>digitalRead()</code>	Lire l'état d'une entrée numérique	<code>digitalRead(broche);</code>	<code>etat = digitalRead(26);</code>
<code>digitalWrite()</code>	Écrire un état logique sur une sortie	<code>digitalWrite(broche, valeur);</code>	<code>digitalWrite(14, HIGH);</code>
<code>analogRead()</code>	Lire une entrée analogique	<code>analogRead(broche);</code>	<code>valeur = analogRead(35);</code>
<code>analogWrite()</code>	Générer une sortie PWM*	<code>analogWrite(broche, valeur);</code>	<code>analogWrite(14, 128);</code>
<code>analogWriteResolution()</code>	Définir la résolution de la sortie PWM*	<code>analogWriteResolution(bits);</code>	<code>analogWriteResolution(8);</code>
<code>analogWriteFrequency()</code>	Définir la fréquence PWM*	<code>analogWriteFrequency(frequence);</code>	<code>analogWriteFrequency(5000);</code>
<code>delay()</code>	Créer une temporisation bloquante	<code>delay(duree);</code>	<code>delay(1000);</code>
<code>delayMicroseconds()</code>	Créer une temporisation en microsecondes	<code>delayMicroseconds(duree);</code>	<code>delayMicroseconds(10);</code>
<code>millis()</code>	Obtenir le temps écoulé depuis le démarrage	<code>millis();</code>	<code>temps = millis();</code>
<code>micros()</code>	Obtenir le temps écoulé en microsecondes	<code>micros();</code>	<code>temps = micros();</code>
<code>constrain()</code>	Limiter une valeur entre deux bornes	<code>constrain(valeur, min, max);</code>	<code>constrain(300, 0, 255);</code>
<code>map()</code>	Changer une valeur d'échelle	<code>map(valeur, min1, max1, min2, max2);</code>	<code>map(2048, 0, 4095, 0, 255);</code>
<code>min()</code>	Retourner la plus petite valeur	<code>min(valeur1, valeur2);</code>	<code>min(10, 20);</code>

API / fonction	Rôle	Syntaxe générique	Exemple
<code>max()</code>	Retourner la plus grande valeur	<code>max(valeur1, valeur2);</code>	<code>max(10, 20);</code>
<code>abs()</code>	Retourner la valeur absolue	<code>abs(valeur);</code>	<code>abs(-25);</code>
<code>random()</code>	Générer un nombre pseudo-aléatoire	<code>random(min, max);</code>	<code>random(0, 100);</code>
<code>randomSeed()</code>	Initialiser le générateur aléatoire	<code>randomSeed(valeur);</code>	<code>randomSeed(analogRead(35));</code>
<code>attachInterrupt()</code>	Associer une fonction à une interruption	<code>attachInterrupt(broche, fonction, mode);</code>	<code>attachInterrupt(26, ISR_bouton, RISING);</code>
<code>detachInterrupt()</code>	Désactiver une interruption	<code>detachInterrupt(broche);</code>	<code>detachInterrupt(26);</code>
<code>Serial.begin()</code>	Initialiser la communication série	<code>Serial.begin(vitesse);</code>	<code>Serial.begin(115200);</code>
<code>Serial.print()</code>	Envoyer une donnée sur le port série	<code>Serial.print(donnee);</code>	<code>Serial.print(valeur);</code>
<code>Serial.println()</code>	Envoyer une donnée suivie d'un retour à la ligne	<code>Serial.println(donnee);</code>	<code>Serial.println("Bonjour");</code>
<code>Serial.available()</code>	Vérifier si des données sont disponibles	<code>Serial.available();</code>	<code>if (Serial.available() > 0)</code>
<code>Serial.read()</code>	Lire un caractère reçu sur le port série	<code>Serial.read();</code>	<code>char c = Serial.read();</code>

Attention : selon la carte utilisée, notamment sur ESP32, les fonctions PWM peuvent différer légèrement selon la version du cœur Arduino-ESP32.

Structure minimale d'un programme

```
void setup() {  
    // put your setup code here, to run once:  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
  
}
```

Différents types de données utilisables avec Arduino.			
TYPE	Valeurs possibles	BITS	Octets
char	-128 à +127	8	1
int	-32768 à +32767	16	2
long	-2147483648 à +2147483647	32	4
Variables Décimales.			
float	$3,4 \times 10^{(-38)}$ à $+3,4 \times 10^{(38)}$	32	4
double	$1,7 \times 10^{(-308)}$ à $+1,7 \times 10^{(308)}$	64	8
La gestion des variables de type double dans Arduino est exactement la même que celle des variables de type float, sans gain de précision.			
Variables NON SIGNÉES : unsigned			
char	0 à 255	8	1
int	0 à 65535	16	2
long	0 à 4294967295	32	4
Variables propres au langage C d'Arduino			
byte	0 à +255	8	1
word	0 à +65535	16	2
boolean	0 ou 1	1	1
Variables booléennes			
boolean Nom Variable = FALSE ou TRUE			
Toute variable peut servir de variable booléenne : • Si elle vaut 0 ce sera interprété comme FALSE. • Si elle est $\neq 0$ ce sera interprété comme TRUE.			
Conversion Numérique analogique pour les entrées A0 à A5			
CAN	0 à 1023	10	2
"boolean" NomBrocheSortieBinaire = LOW ou HIGH			

```
int valeur;
float temperature;
bool etat;
const int LED = 8;
```

Structures de contrôle

```
if (...)
{
```

```
}  
else  
{  
}  
  
for (...)  
{  
}  
  
while (...)  
{  
}
```

Documents Ressources.

https://docs.arduino.cc/language-reference/?utm_source